SAMS Developer & Operations Book — Denis

Smart Attendance Management System — production architecture, failure modes, and Super Admin AI troubleshooting reference

Item	Value
Version	1.0.0
Last updated	2026-06-04
Author / owner	Denis Macharia (Super Admin)
Production VPS	`182.143.228.182`
Deploy path	`/var/www/sams`
PM2 process	`sams-api`
API port (localhost)	`3001`
Main app	https://app.smart-managment.com
Super Admin	https://super.smart-managment.com (school code `SUPERADMIN`)
API (direct)	https://api.smart-managment.com
Super Admin login	`admin@smart-managment.com` / code `SUPERADMIN`
Companion (quick fixes)	`docs/SAMS-OPS-RUNBOOK.md`
Product docs	`DOCUMENTATION.md`

How to use this book

This is the **deep companion** to `docs/SAMS-OPS-RUNBOOK.md`. The runbook gives symptom → command shortcuts. This book explains **why** things work, **what** breaks them, **where** to look in the repo and on the VPS, and **how** to fix and prevent recurrence.

Each major section follows: **Architecture** → **Failure modes** → **What to check** → **Commands** → **Fix** → **Prevention**.

Super Admin AI loads this file automatically (see §16). When troubleshooting with AI, cite symptoms explicitly; the AI can execute platform actions (suspend/unsuspend, stats, license) but cannot read live `.env` or SSH into the server — you run commands on the VPS.

Table of contents

---

1. System architecture overview

1.1 Monorepo layout

```
/var/www/sams/                          # Production root (git clone)
├── packages/
│   ├── shared/                         # @sams/shared — types, enums, license encoding
│   ├── backend/                        # @sams/backend — Express API (port 3001)
│   │   ├── src/                        # TypeScript source
│   │   ├── dist/                       # Compiled JS (PM2 runs this)
│   │   ├── prisma/                     # schema + migrations
│   │   ├── bin/pm2-start.js            # PM2 entry — loads .env then boots dist
│   │   └── .env                        # JWT, DATABASE_URL, PORT (NOT in git)
│   ├── frontend/                       # @sams/frontend — main school app SPA
│   │   └── dist/                       # nginx serves this for app.*
│   └── super-admin/                    # @sams/super-admin — platform admin SPA
│       └── dist/                       # nginx serves this for super.*
├── nginx/sams.conf                     # Template — copied to /etc/nginx/sites-enabled/
├── ecosystem.config.js                 # PM2 app definition
├── secrets/
│   └── providers.env                   # AI, SMS, SMTP, M-Pesa keys (NOT in git)
├── scripts/                            # deploy, restart, backup, verify
├── uploads/                            # Avatars (default /var/www/sams/uploads)
└── docs/
    ├── SAMS-OPS-RUNBOOK.md             # Quick ops blueprint
    └── SAMS-DEVELOPER-OPS-BOOK-DENIS.md  # This file
```

1.2 Request flow (browser → response)

```
Browser (app.smart-managment.com or super.smart-managment.com)
    │
    ▼
Cloudflare (DNS, TLS edge, real IP via CF-Connecting-IP)
    │
    ▼
nginx :443 (nginx/sams.conf → /etc/nginx/sites-enabled/sams.conf)
    │
    ├── Static SPA: try_files → packages/{frontend|super-admin}/dist/index.html
    ├── /uploads/ → alias /var/www/sams/uploads/
    ├── /api/* → proxy_pass http://127.0.0.1:3001
```

```
    └── /socket.io/* → WebSocket upgrade to 127.0.0.1:3001
    │
    ▼
PM2 process `sams-api` (ecosystem.config.js)
    │
    └── packages/backend/bin/pm2-start.js
            ├── load packages/backend/.env
            ├── overlay secrets/providers.env (and .env.secrets, ai.env)
            └── require(dist/index.js).boot()
    │
    ▼
Express (packages/backend/src/registerApplication.ts)
    │
    ├── Global middleware (CORS, JSON, rate limits)
    ├── /health — DB + Redis + AI/SMS/OTP status
    ├── /api/v1/* routes (auth, sessions, attendance, ai, super, …)
    ├── authenticate → enforceSchoolScope → licenseGuard (most routes)
    └── Socket.io (attendance live updates)
    │
    ├── PostgreSQL via Prisma (packages/backend/src/lib/prisma.ts)
    └── Redis via ioredis (packages/backend/src/lib/redis.ts)
```

Key files:

Layer	File
HTTP boot	`packages/backend/src/index.ts`
Route registration	`packages/backend/src/registerApplication.ts`
PM2 env load	`packages/backend/bin/pm2-start.js`, `packages/backend/bin/load-env-from-file.js`
nginx template	`nginx/sams.conf`
PM2 config	`ecosystem.config.js`

1.3 Startup sequence (why `/health` matters)

1. `pm2-start.js` loads env synchronously **before** Node requires the app.
2. `boot()` in `index.ts` calls `validateProductionSecrets()` — **throws if JWT/QR secrets < 64 chars** in production.
3. HTTP server listens; `/health` returns `503 starting` until `connectDependencies()` finishes.
4. Redis `connect()` + Prisma `$connect()` → `setApiReady(true)`.
5. Background jobs start: QR refresh (`jobs/qrRefresh.ts`), notifications (`jobs/notifications.ts`).
6. `registerApplication()` mounts routes and full `/health` with AI/SMS blocks.

First response on any outage:

```bash
cd /var/www/sams
pm2 status
curl -sS http://127.0.0.1:3001/health | jq .
sudo nginx -t
```

`/health` field	Meaning
`checks.database: true`	PostgreSQL reachable
`checks.redis: true`	Redis PONG
`ai.configured: true`	Primary AI key loaded
`sms.configured: true`	AT_API_KEY present
`sms.mode: "production"`	Real SMS (not sandbox)
`otp.loginEnabled`	OTP at login (usually false)
`otp.passwordResetEnabled`	Forgot-password SMS

If local `/health` fails → **backend** (PM2, env, DB, Redis).
If local OK but browser fails → **nginx**, **dist missing**, or **Cloudflare/DNS**.

1.4 API route map (backend)

Prefix	Router file	Purpose
`/api/v1/auth`	`routes/auth.ts`	Login, refresh, OTP, WebAuthn
`/api/v1/sessions`	`routes/sessions.ts`	Attendance sessions
`/api/v1/attendance`	`routes/attendance.ts`	Marking, QR scan
`/api/v1/ai`	`routes/ai.ts`	Chat, image upload, voice
`/api/v1/super`	`routes/superAdmin.ts`	Platform admin API
`/api/v1/knowledge`	`routes/knowledge.ts`	AI knowledge base CRUD
`/api/v1/notifications`	`routes/notifications.ts`	In-app + SMS notifications
`/api/v1/biometric`	`routes/biometric.ts`	Face enrollment/verify

Public paths (no JWT): login, refresh, activate, payment callback, guest AI query — see `PUBLIC_PATHS` in `registerApplication.ts`.

---

2. Environment & secrets

2.1 How env loading works

Load order (later wins):

1. `packages/backend/.env` — core app config
2. `packages/backend/.env.secrets` — optional local overlay
3. `secrets/providers.env` or `/var/www/sams/secrets/providers.env`
4. Legacy `secrets/ai.env` (migrate to providers.env)

Implementation:

Context	File
PM2 runtime	`packages/backend/bin/load-env-from-file.js`
Shell scripts	`scripts/lib/merged-env.sh` (`read_merged_env`, `source_merged_env`)
TypeScript (dev)	`packages/backend/src/config/loadEnv.ts`

`FORCE_FROM_FILE` keys (`JWT_SECRET` , `JWT_REFRESH_SECRET` , `QR_SECRET` , `DATABASE_URL`) always prefer `.env` when non-weak — prevents PM2 inherited junk from overriding real secrets.

2.2 File purposes

File	Contains	In git?
`packages/backend/.env`	`DATABASE_URL`, `JWT_*`, `QR_SECRET`, `PORT`, `CORS_ORIGIN`, `APP_URL`	**No**
`secrets/providers.env`	`OPENAI_*`, `AT_*`, `SMTP_*`, `MPESA_*`, `CONVERSATION_MASTER_KEY`, `BIOMETRIC_MASTER_KEY`	**No**
`secrets/providers.env.example`	Template with placeholders	Yes
`.env.example`	Dev template	Yes

Generate / repair production `.env` :

```
cd /var/www/sams
bash scripts/set-production-env.sh   # JWT, QR, APP_URL, OTP flags
bash scripts/verify-secrets.sh
```

Write AI keys interactively:

```
bash scripts/write-providers-env.sh
```

2.3 Common corruption patterns

Pattern A: Junk line in `providers.env`

Symptom: `pm2 status` empty after deploy; shell errors when sourcing env.

Cause: Pasted command like `bash scripts/restart-api.sh#` (trailing `#` breaks path) into secrets file. Old `source .env` would execute it; `safe_source_env_file` in `merged-env.sh` now **ignores** non `KEY=value` lines, but manual `source` or typos still cause harm.

Check:

```
grep -n "restart-api\|bash \|pm2 \|curl " secrets/providers.env packages/backend/.env
cat -A secrets/providers.env | head -30   # reveal hidden chars
```

Fix:

```
bash scripts/fix-providers-env.sh
bash scripts/verify-secrets.sh
bash scripts/restart-api.sh
```

Pattern B: Doubled quotes on `REDIS_URL`

Symptom: Logs show `%22redis://localhost:6379%22`, Redis connection errors.

Cause: `REDIS_URL="redis://localhost:6379"` pasted with extra quotes, or export chain doubling quotes.

Fix: Single unquoted or properly quoted line:

```
REDIS_URL=redis://localhost:6379
```

`packages/backend/src/lib/redis.ts` strips outer quotes at runtime, but malformed values still break if nested.

Pattern C: Weak JWT after git reset

Symptom: PM2 restart loop; log `[STARTUP] JWT_SECRET must be … 64+ chars`.

Cause: `.env` was tracked in git with placeholders; deploy restores backup but empty backup = weak secrets.

Fix:

```
bash scripts/set-production-env.sh
bash scripts/restart-api.sh
```

Deploy protects env via `backup_deploy_env_files` / `restore_deploy_env_files` in `scripts/deploy-production.sh`.

Pattern D: `AT_USERNAME="SAMS"` confusion

Symptom: SMS config looks set but sends fail.

Cause: `AT_USERNAME` must be Africa's Talking **dashboard app username** (e.g. `deno1`), **not** sender ID. Sender ID is `AT_SENDER_ID`.

Fix: `bash scripts/configure-production-at.sh` or edit manually; `fix-providers-env.sh` detects this mistake.

2.4 Verification commands

```
cd /var/www/sams
bash scripts/verify-secrets.sh           # all providers
bash scripts/verify-secrets.sh --ai-only
bash scripts/test-pm2-env-load.js      # simulates pm2-start env merge
bash scripts/diagnose-api.sh
```

---

3. Deployment pipeline

3.1 `scripts/deploy-production.sh` step-by-step

Step	What it does	What can break
Node version check	Warns if Node < 20	Old Node → build/runtime failures
`backup_deploy_env_files`	Copies `.env`, `providers.env` to temp	—
`git fetch && git reset --hard origin/main`	Clean match to GitHub main	Loses uncommitted tracked changes
`restore_deploy_env_files`	Restores protected env files	If backup empty, weak secrets return
`npm ci`	Clean install	Network, lockfile mismatch
`prisma generate`	Client codegen	Schema errors
Build shared → backend → frontend → super-admin	Fresh `dist/`	TypeScript errors, OOM
`verify_*_dist`	Checks index.html, JS bundles, `/health` in dist	Stale backend dist
`baseline-prisma-init.sh`	Marks init migration on existing DB	P3018 if skipped on legacy DB
`prisma migrate deploy`	Applies pending migrations	SQL conflicts, DB down
`create-super-admin`	Ensures super admin exists	Non-fatal (`
JWT strength check	Fails deploy if weak	Missing `set-production-env.sh`
`pm2 delete sams-api` + start	Fresh PM2 with new env	Env corruption → empty PM2
`nginx -t && reload`	Applies config	Bad nginx syntax
`post-deploy-verify.sh`	Smoke tests	Fails deploy exit 1

Full deploy:

```
cd /var/www/sams
bash scripts/backup-production.sh
bash scripts/deploy-production.sh
```

GitHub Actions: Push to `main` triggers CI tests, then SSH runs the same deploy script on the VPS.

3.2 What deploy never touches

- `packages/backend/.env`
- `secrets/providers.env`
- `packages/backend/.env.secrets`
- `/var/www/sams/uploads/`

Comment in deploy script: "*Does NOT modify packages/backend/.env, .env.secrets, or secrets/providers.env*".

3.3 Post-deploy verification

`scripts/post-deploy-verify.sh` checks:

- Node ≥ 20
- All three `dist/` artifacts exist
- PM2 `sams-api` online
- `/health` HTTP 200 with database + redis
- AI/SMS configured flags (warn if not)
- Optional `VERIFY_AI_QUERY=1` for live AI smoke

```
VERIFY_AI_QUERY=1 bash scripts/post-deploy-verify.sh
bash scripts/smoke-production.sh
```

Local dev (API on `PORT=3001`):

```
npm run dev -w @sams/backend   # separate terminal
bash scripts/smoke-test-local.sh
VERIFY_LOGIN_IDENTIFIER=teacher@school.com VERIFY_LOGIN_PASSWORD='***' bash scripts/smoke-test-local.sh
```

3.4 Partial deploy (UI only)

```
cd /var/www/sams
git pull origin main
npm run build -w @sams/frontend
npm run build -w @sams/super-admin
sudo nginx -t && sudo systemctl reload nginx
```

Backend unchanged — no PM2 restart needed unless API code changed.

---

4. PM2 & Node

4.1 How PM2 runs SAMS

`ecosystem.config.js`:

- **Name:** `sams-api`
- **Script:** `packages/backend/bin/pm2-start.js` (not raw `dist/index.js`)
- **CWD:** repo root `/var/www/sams`
- **Mode:** `fork`, 1 instance
- **Logs:** `/var/log/sams/sams-api-out.log`, `sams-api-error.log`
- **Memory:** restart at 512M
- **wait_ready:** `false` (avoid crash loops on some PM2 builds)

`pm2-start.js` loads env, finds `dist/index.js`, calls `boot()`.

4.2 Safe restart

Always prefer:

```
cd /var/www/sams
bash scripts/restart-api.sh
```

This script:

1. Validates JWT via merged env
2. `source_merged_env` then `pm2 delete` + `pm2 start`
3. Waits up to 60s for `/health` HTTP 200
4. On failure: verbose curl + last 30 PM2 log lines

After env-only change:

```
bash scripts/restart-api.sh
# or:
pm2 reload ecosystem.config.js --env production --update-env
pm2 save
```

4.3 PM2 empty after deploy

Symptom: `pm2 describe sams-api` → not found.

Causes:

1. Junk in `providers.env` broke shell `source` during deploy (see §2.3)
2. Deploy exited before PM2 start (JWT check failed, build failed)
3. PM2 not saved / different Linux user

Fix:

```
cd /var/www/sams
bash scripts/fix-providers-env.sh    # if env suspect
bash scripts/restart-api.sh
pm2 save
pm2 startup   # once per server — follow printed command
```

4.4 Node version

SAMS requires **Node 20+** (`.nvmrc`).

```
node -v
which node
bash scripts/upgrade-node20.sh          # nvm
bash scripts/install-node20-ubuntu.sh   # apt NodeSource
```

Mismatch (PM2 uses old Node while shell uses nvm) → cryptic build or runtime errors.

4.5 Crash loop diagnosis

```
pm2 logs sams-api --err --lines 100 --nostream
pm2 logs sams-api --lines 100 --nostream
grep -E "STARTUP|JWT|Error|ECONNREFUSED" /var/log/sams/sams-api-error.log | tail -50
```

Common log lines:

Log	Meaning
`[STARTUP] JWT_SECRET must be …`	Run `set-production-env.sh`
`[Redis] Error:`	Redis down or bad URL
`Backend build not found`	Missing `dist/` — rebuild backend
`EADDRINUSE :3001`	Another process on port 3001

---

5. Nginx

5.1 Server blocks (`nginx/sams.conf`)

`server_name`	Serves	`root` / proxy
`app.smart-managment.com`	Main SPA + `/api/` + `/socket.io/` + `/uploads/`	`packages/frontend/dist`
`api.smart-managment.com`	Direct API + socket	proxy → `:3001`
`super.smart-managment.com`	Super Admin SPA + `/api/`	`packages/super-admin/dist`
`smart-managment.com`, `www`	301 → app	—

Production copy:

```
sudo cp /var/www/sams/nginx/sams.conf /etc/nginx/sites-enabled/sams.conf
sudo nginx -t
sudo systemctl reload nginx
```

5.2 `client_max_body_size 25m`

Set on **all three** HTTPS server blocks (app, api, super) at lines ~76, ~159, ~208 in repo template.

Symptom: HTTP **413** on photo upload (AI image, avatar, biometric).

Cause: Old nginx config had default 1m; upload hits nginx **before** Express multer (5MB per file in `routes/ai.ts`).

Check:

```
grep -n client_max_body_size /etc/nginx/sites-enabled/sams.conf
# Expect 3 lines at 25m
```

Fix: Copy repo config (above), reload nginx. Rebuild SPAs if cached old error pages.

5.3 502 Bad Gateway

Cause chain:

1. PM2 not running → nothing on `:3001`
2. API crash loop → intermittent 502
3. nginx upstream timeout (60s on API locations)

Check:

```
curl -v http://127.0.0.1:3001/health
sudo tail -50 /var/log/nginx/error.log
```

5.4 Rate limiting

- Login: `5r/m` on `app` → `/api/v1/auth/login`
- API subdomain: `30r/s` with burst 50

False 429s under load → check Cloudflare rate rules too.

5.5 SSL

Certificates: `/etc/letsencrypt/live/smart-managment.com/`

```
sudo certbot certificates
sudo certbot renew --dry-run
```

---

6. PostgreSQL & Prisma

6.1 Schema location

- Schema: `packages/backend/prisma/schema.prisma`
- Migrations: `packages/backend/prisma/migrations/`
- Baseline init: `20240101000000_init`

6.2 Connection string

In `packages/backend/.env`:

```
DATABASE_URL=postgresql://USER:PASSWORD@localhost:5432/sams?schema=public
```

Check:

```
grep DATABASE_URL packages/backend/.env
sudo systemctl status postgresql
cd /var/www/sams/packages/backend && npx prisma db execute --stdin <<< 'SELECT 1'
```

6.3 Migrations in production

Deploy runs:

```
bash scripts/baseline-prisma-init.sh
```

```
npx prisma migrate deploy
```

Never use `prisma migrate dev` on production (creates shadow DB).

6.4 P3018 / "PlanTier already exists"

Cause: Database existed before baseline migration `20240101000000_init` was added. Prisma tries to re-run init SQL → duplicate enum/tables.

Symptoms:

- Deploy fails at `prisma migrate deploy`
- Error mentions `P3018` or `PlanTier already exists`

Fix:

```
cd /var/www/sams
bash scripts/baseline-prisma-init.sh
cd packages/backend
npx prisma migrate resolve --rolled-back 20240101000000_init 2>/dev/null || true
npx prisma migrate resolve --applied 20240101000000_init
npx prisma migrate deploy
```

`baseline-prisma-init.sh` detects existing `User` table and marks init as applied without re-running SQL.

6.5 `db push` vs `migrate deploy`

Command	Use
`migrate deploy`	**Production** — applies versioned migrations
`db push`	Dev only — can drift schema without migration history

6.6 Prisma Studio (careful on prod)

```
cd /var/www/sams/packages/backend
npx prisma studio
# Binds locally — use SSH tunnel, not public exposure
```

---

7. Redis

7.1 Role in SAMS

- Session/event cache patterns
- Rate limiting backing store (via middleware)
- Health check: `redis.ping()` in `/health`

Client: `packages/backend/src/lib/redis.ts` — `lazyConnect: true`, strips quotes from URL.

7.2 Failure modes

Symptom	Cause

`checks.redis: false`	redis-server stopped
`%22redis://...%22` in errors	Malformed `REDIS_URL`
`Connection is closed`	Redis restarted while API running — usually recovers

Fix:

```
grep REDIS_URL packages/backend/.env secrets/providers.env
redis-cli ping                    # expect PONG
sudo systemctl start redis-server
sudo systemctl enable redis-server
bash scripts/restart-api.sh
```

Good startup log: `[Redis] Connected` then `Database and Redis connected — API ready`.

7.3 Default

If `REDIS_URL` unset → `redis://localhost:6379`.

---

8. Authentication & sessions

8.1 JWT model

Implementation: `packages/backend/src/services/authService.ts`

Token	Expiry	Storage
Access JWT	15 minutes	Client memory / header
Refresh JWT	30 days	Hashed in DB

Secrets: `JWT_SECRET`, `JWT_REFRESH_SECRET` (64+ chars production).

Payload includes: `sub`, `schoolId`, `role`, optional `departmentId`, `classId`.

8.2 Login flow

1. Client POST `/api/v1/auth/login` with `schoolCode`, `identifier`, `password`
2. Find school by code; find user by email/admission/phone within school
3. Bcrypt verify; lockout after 5 failures / 15 min
4. Issue token pair; audit `USER_LOGIN`

Super Admin: school code `SUPERADMIN`, user `admin@smart-managment.com`.

8.3 Middleware chain

For protected routes (`registerApplication.ts`):

```
authenticate → enforceSchoolScope → licenseGuard
```

Middleware	File	Blocks when

`authenticate`	`middleware/auth.ts`	Invalid/missing JWT
`enforceSchoolScope`	`middleware/rbac.ts`	Cross-tenant access
`licenseGuard`	`middleware/licenseGuard.ts`	School suspended

8.4 School suspension

`licenseGuard.ts` :

- `SUPER_ADMIN` → always pass
- Platform school code `SAMS_PLATFORM` → pass
- `school.isSuspended === true` → **403** `SCHOOL_SUSPENDED`

Suspension also revokes sessions via `licenseService.suspendSchool()` .

Unsuspend paths:

1. Super Admin panel → Schools → Unsuspend
2. Super Admin AI: `unsuspend school [name]` (see §9)
3. Direct DB: `UPDATE "School" SET "isSuspended" = false WHERE …`

8.5 OTP flags

Set in `.env` / `set-production-env.sh` :

```
OTP_LOGIN_ENABLED=false
OTP_PASSWORD_RESET_ENABLED=true
```

Requires working SMS for password reset OTP.

---

9. Super Admin AI

9.1 Architecture

```
POST /api/v1/ai/query (authenticated Super Admin)
    │
    ▼
aiService.ts → openaiEngine.ts
    │
    ├── buildSystemPrompt()
    │     ├── Role scope text
    │     ├── getSystemDocumentationExcerpt(role)  ← runbook + this book + DOCUMENTATION.md
    │     ├── Custom AI Knowledge (DB)
    │     ├── Real-time stats (school counts)
    │     └── buildRoleActionsPromptSection()
    │
    ├── actionIntentDetector.detect(message, role)
    │     ├── Regex fast path (roleActionRegistry patterns)
    │     └── LLM fallback (llmActionClassifier.ts)
    │
    └── Execute handler → return answer
```

Key files:

File	Purpose
`services/ai/openaiEngine.ts`	Prompt build, LLM call
`services/ai/systemDocumentation.ts`	Loads markdown into context
`services/ai/actionIntentDetector.ts`	Action vs question
`services/ai/roleActionRegistry.ts`	Per-role action list
`services/ai/handlers/superAdminHandlers.ts`	Suspend, license, stats, …
`services/ai/aiProviderConfig.ts`	OpenRouter/Groq clients, health
`routes/ai.ts`	HTTP endpoints, multer for images

9.2 AI provider config

Primary (typical production): **OpenRouter** via `OPENAI_API_KEY`, `OPENAI_BASE_URL`, `OPENAI_MODEL`.

Fallback: **Groq** via `OPENAI_FALLBACK_*`.

Vision (images): `VISION_MODEL` on OpenRouter — Groq fallback is **text-only**.

Verify:

```
bash scripts/verify-secrets.sh --ai-only
bash scripts/diagnose-ai.sh
curl -sS 'http://127.0.0.1:3001/health?ai_probe=1' | jq .ai
```

Keys live in `secrets/providers.env`, not git.

9.3 Action routing — suspend / unsuspend

Definitions: `packages/backend/src/services/ai/handlers/superAdminHandlers.ts`

Unsuspend patterns (regex fast path):

- `unsuspend [school name]`
- `unblock [school name]`
- `reactivate [school name]`
- `enable [school name]`
- `undo suspension for [school]`
- `undo suspension of [school]`

Suspend patterns (destructive — requires "yes" confirmation):

- `suspend [school name]`
- `block [school name]`
- `disable [school name]`

Past failure — AI unsuspend regex: If natural language like "please undo the suspension for XYZ Academy" failed, patterns were expanded to include `undo suspension for/of` variants (see `superAdminActions` array ~line 277). Prefer explicit: **`unsuspend school XYZ Academy`**.

School name matching: Case-insensitive `contains` on `School.name` — partial names work but ambiguous if multiple matches.

9.4 Image upload

Endpoint: `POST /api/v1/ai/query-with-image` (`routes/ai.ts`)

- Up to 4 images, 5 MB each (multer)
- nginx must allow 25m body (§5.2)
- Requires `VISION_MODEL` configured

Symptom: "Could not send that photo" / 413 → nginx fix, not AI keys.

9.5 Knowledge base

- DB model: `AIKnowledge` via `services/knowledgeService.ts`
- API: `/api/v1/knowledge` , Super Admin UI
- Merged into prompt after system docs
- Category `ops` for runbook excerpts (manual supplement)

9.6 Super Admin vs school AI

Panel	URL	Scope
Super Admin AI	super.smart-managment.com	All schools, licenses, suspend, system stats
School AI	app.smart-managment.com	Single school; no platform actions

9.7 Conversation memory

Requires `CONVERSATION_MASTER_KEY` (32+ chars) in `providers.env` . Without it, encrypted thread memory disabled (warn at startup).

9.8 School AI role actions

School AI action execution is centralized in `services/ai/roleActionRegistry.ts` and routed through `AIService.executeAction()` , so route/service RBAC is still the final authority.

Role	Current action surface
SCHOOL_ADMIN	school/class/department in-app notifications, registration links, user/class/department actions, password reset, school stats
HOD	department/class in-app notifications, department stats, registration links, teacher assignment, department class creation, department attendance session start/end/manual marking
TEACHER	class in-app messages, registration links, session start/end, manual attendance marking, class roster
STUDENT	own attendance/timetable/teachers/HOD/class/department/class-rep info and reminders

Attendance AI actions must follow timetable/session rules: HODs are scoped to their department classes; teachers are scoped to taught classes. Destructive actions require confirmation where configured.

---

10. Attendance, biometric, notifications

10.1 Attendance sessions

Component	File
Service	`services/attendanceService.ts`
Routes	`routes/sessions.ts`, `routes/attendance.ts`
QR job	`jobs/qrRefresh.ts` — 30s token rotation
Live updates	`sockets/attendanceSocket.ts`

Failure modes:

- QR expired → student scanned old code; refresh interval 30s
- GPS rejection → location outside school geofence
- Session not active → teacher must start session first
- WebSocket disconnect → client falls back to polling

Additional session guards:

- No current timetable slot -> session start is denied; teacher/HOD must use the scheduled class or fix timetable first.
- HOD session access -> routes and sockets allow only sessions whose class belongs to the HOD department.

10.2 Biometric

Component	File
Service	`services/biometricService.ts`
Encryption	`services/biometricEncryption.ts`
Routes	`routes/biometric.ts`
Key	`BIOMETRIC_MASTER_KEY` in providers.env

Production gate: `scripts/production-readiness-check.sh` fails if biometric key missing or routes not in dist.

10.3 Notifications

Component	File
Service	`services/notificationService.ts`
Scoped send	`services/scopedNotificationSend.ts`
Job	`jobs/notifications.ts`
Routes	`routes/notifications.ts`

In-app notifications work without SMTP. SMS uses Africa's Talking (§11).

Class rep replies: Students with `isClassRep: true` can reply to teacher messages — scope enforced in notification routes.

Current app-only mode: The frontend notification composer is app-only. SMS is reserved for OTP/password-reset/app onboarding flows until live AT capacity is intentionally re-enabled. Notification attachments are stored under `uploads/notifications` and served through authenticated `/api/v1/notifications/attachments/:id`, so sender and recipient can open/download images, PDFs, and files.

11. SMS / Africa's Talking

11.1 Configuration

File: `packages/backend/src/config/africasTalking.ts`

Variable	Meaning
`AT_API_KEY`	API key (`atsk_...`)
`AT_USERNAME`	Dashboard **app username** (prod: e.g. `deno1`; sandbox: `sandbox`)
`AT_SENDER_ID`	Approved sender label (e.g. `SAMS`)

Sandbox vs production:

- `AT_USERNAME=sandbox` → `sms.mode: sandbox` in `/health`
- Production requires real username + production API key
- `verify-secrets.sh` / readiness scripts warn, not fail, on sandbox SMS when app-only mode disables all SMS-dependent production features

App-only exception: when `ready-app-only-production.sh` disables all SMS-dependent production features (`OTP_LOGIN_ENABLED=false`, `OTP_PASSWORD_RESET_ENABLED=false`, notification SMS hidden in frontend), readiness scripts warn on sandbox SMS instead of failing critical checks.

11.2 InvalidSenderId

Symptom: SMS API returns `InvalidSenderId`; OTP never arrives.

Cause: `AT_SENDER_ID=SAMS` not yet approved in Africa's Talking dashboard. Config can be **correct** but carrier rejects until approved.

Check:

```
curl -sS http://127.0.0.1:3001/health | jq .sms
bash scripts/verify-secrets.sh
```

Fix:

1. Request sender ID approval in AT dashboard
2. Until approved: keep `OTP_LOGIN_ENABLED=false`; password reset SMS will fail gracefully
3. `bash scripts/configure-production-at.sh` for guided setup

Common mistake: `AT_USERNAME=SAMS` — wrong field; username ≠ sender ID.

11.3 Services using SMS

- `otpService.ts` - login/reset OTP when enabled
- `passwordResetService.ts` - forgot password flow when SMS/email reset is enabled

- `phoneOnboardingService.ts` - phone verification/welcome modes
- `notificationService.ts` - still has backend SMS primitives, but the frontend notification composer is app-only until live SMS capacity is intentionally re-enabled

---

Current production note: `notificationService.ts` still has backend SMS primitives, but the frontend notification composer is app-only until live SMS capacity is intentionally re-enabled. OTP/password-reset/app onboarding are the SMS-dependent paths to verify before turning live SMS back on.

12. Frontend & Super Admin builds

12.1 Build commands

```
cd /var/www/sams
npm run build -w @sams/shared      # dependency for others
npm run build -w @sams/backend
npm run build -w @sams/frontend
npm run build -w @sams/super-admin
```

Deploy always rebuilds all four — **never rely on git for dist/**.

12.2 Output paths

App	dist	nginx root
Main	`packages/frontend/dist/`	app.smart-managment.com
Super Admin	`packages/super-admin/dist/`	super.smart-managment.com

Verify:

```
ls -la packages/frontend/dist/index.html
ls -la packages/super-admin/dist/assets/*.js | head
```

12.3 Environment at build time

Vite embeds `import.meta.env.VITE_*` at **build** time. Production API URL typically same-origin `/api` on app/super domains — rebuild after changing Vite env.

Backend env (`.env`) is runtime-only — no rebuild needed for JWT/DB changes.

12.4 Stale backend dist

Deploy checks `registerApplication.js` contains `getAIHealthSummary` — stale dist missing expanded `/health` :

```
rm -rf packages/backend/dist
npm run build -w @sams/backend
bash scripts/restart-api.sh
```

---

13. Backups & recovery

13.1 Create backup

```
cd /var/www/sams
bash scripts/backup-production.sh              # secrets + DB dump
bash scripts/backup-production.sh --with-uploads # + avatars
ls -la backups/production-*
```

Output: `backups/production-YYYYMMDD-HHMMSS/`

File	Contents
`providers.env`	Provider secrets copy
`backend.env.snapshot`	JWT + DATABASE_URL
`database.dump`	`pg_dump -Fc`
`uploads/`	Optional tarball
`RESTORE.md`	Restore instructions

Secrets-only:

```
bash scripts/backup-secrets.sh
# → secrets/providers.env.backup.TIMESTAMP
```

13.2 Download off-server

From Windows PowerShell:

```
scp -r root@182.143.228.182:/var/www/sams/backups/production-YYYYMMDD-HHMMSS
"$env:USERPROFILE\Desktop\SAMS-backups\"
```

13.3 Restore (summary)

1. Stop API: `pm2 stop sams-api`
2. Restore `providers.env` and `packages/backend/.env` from backup
3. Restore DB: `pg_restore -d sams -c database.dump` (see `RESTORE.md` in backup folder)
4. Restore uploads if needed
5. `bash scripts/restart-api.sh`
6. `bash scripts/post-deploy-verify.sh`

Before risky edits:

```
bash scripts/backup-secrets.sh
```

---

14. Logs & debugging

14.1 Log locations

Log	Path / command
PM2 stdout	`/var/log/sams/sams-api-out.log`
PM2 stderr	`/var/log/sams/sams-api-error.log`
Live tail	`pm2 logs sams-api`
nginx access	`/var/log/nginx/access.log`
nginx error	`/var/log/nginx/error.log`
PostgreSQL	`/var/log/postgresql/postgresql-*-main.log`

14.2 Useful grep patterns

```
# Startup / secrets
grep -E "STARTUP|JWT|SAMS\]" /var/log/sams/sams-api-error.log | tail -30

# Redis
grep -i redis /var/log/sams/sams-api-error.log | tail -20

# AI errors
grep -iE "openai|groq|openrouter|AI\]" /var/log/sams/sams-api-error.log | tail -30

# SMS
grep -iE "africa|InvalidSender|SMS" /var/log/sams/sams-api-out.log | tail -20

# Prisma
grep -i prisma /var/log/sams/sams-api-error.log | tail -20

# nginx 413/502
grep -E "413|502|upstream" /var/log/nginx/error.log | tail -20
```

14.3 Diagnostic scripts

```
bash scripts/diagnose-api.sh
bash scripts/diagnose-ai.sh
bash scripts/diagnose-login.js    # local / with env
bash scripts/production-readiness-check.sh
```

14.4 Health deep probe

```
curl -sS 'http://127.0.0.1:3001/health?ai_probe=1' | jq .
```

Runs live AI provider probe when configured (12s timeout).

---

15. Security & rotation

15.1 If exposed — rotate immediately

Secret	Impact	Action
`JWT_SECRET` / `JWT_REFRESH_SECRET`	Forge tokens	`set-production-env.sh`, restart, all users re-login
`OPENAI_*` / Groq keys	API abuse	Regenerate in provider dashboard, update providers.env
`AT_API_KEY`	SMS abuse	Regenerate in AT dashboard
`DATABASE_URL` password	DB breach	Change PG password, update .env
`MPESA_*`	Payment fraud	Rotate in Safaricom portal
`CONVERSATION_MASTER_KEY`	Decrypt old AI threads	Rotate with `CONVERSATION_MASTER_KEY_PREVIOUS` if supported
`BIOMETRIC_MASTER_KEY`	Biometric data	Requires re-enrollment plan

15.2 Rotation procedure

```
bash scripts/backup-secrets.sh
# edit secrets/providers.env and/or packages/backend/.env
bash scripts/verify-secrets.sh
bash scripts/restart-api.sh
bash scripts/post-deploy-verify.sh
# Force logout: JWT rotation invalidates access tokens in ~15m; refresh tokens until DB purge
```

15.3 AI must never expose

Super Admin AI prompt instructs: never output API keys, JWT secrets, DB passwords. Documentation excerpts are sanitized templates only.

15.4 File permissions

```
chmod 600 packages/backend/.env secrets/providers.env
chmod 700 secrets/
```

---

16. Quick reference tables

16.1 Symptom → cause → file → command

Symptom	Likely cause	Check file/folder	Fix command
Site down / 502	PM2 stopped	`pm2 status`, `/var/log/sams/`	`bash scripts/restart-api.sh`

PM2 empty after deploy	Junk in providers.env	`secrets/providers.env`	`bash scripts/fix-providers-env.sh && bash scripts/restart-api.sh`
JWT crash loop	Weak JWT_SECRET	`packages/backend/.env`	`bash scripts/set-production-env.sh`
Redis `%22redis%22`	Quoted REDIS_URL	`.env`	Fix line, `systemctl start redis-server`, restart API
P3018 / PlanTier exists	Baseline not applied	`prisma/migrations/`	`bash scripts/baseline-prisma-init.sh`
Photo 413	nginx body size	`/etc/nginx/sites-enabled/sams.conf`	Copy `nginx/sams.conf`, reload
AI not configured	Missing OPENAI_*	`secrets/providers.env`	`write-providers-env.sh`, verify, restart
AI won't unsuspend	Regex miss / wrong phrasing	`handlers/superAdminHandlers.ts`	Say `unsuspend school [exact name]`
SMS InvalidSenderId	Sender not approved	AT dashboard	Wait for approval; check AT_USERNAME
SCHOOL_SUSPENDED 403	School flagged	DB `School.isSuspended`	Super Admin unsuspend
Blank SPA	Missing dist	`packages/*/dist/`	`npm run build -w @sams/frontend`
Migration failed	Pending SQL error	`prisma/migrations/`	Fix DB, `migrate deploy`
WebSocket fail	nginx upgrade	`nginx/sams.conf` `/socket.io/`	Reload nginx, check proxy headers

16.2 Script index

Script	Purpose
`deploy-production.sh`	Full production deploy
`restart-api.sh`	Safe PM2 restart + health wait
`backup-production.sh`	Full backup
`backup-secrets.sh`	Providers backup only
`verify-secrets.sh`	Validate merged env
`fix-providers-env.sh`	Repair corrupted providers.env
`set-production-env.sh`	JWT/QR/APP_URL bootstrap
`baseline-prisma-init.sh`	P3018 baseline

`post-deploy-verify.sh`	Smoke after deploy
`configure-production-at.sh`	Africa's Talking setup
`production-readiness-check.sh`	Go-live gate
`diagnose-api.sh` / `diagnose-ai.sh`	Targeted diagnostics

16.3 Super Admin AI context loading

After deploy, `packages/backend/src/services/ai/systemDocumentation.ts` loads for `SUPER_ADMIN` role:

1. **Operations Runbook** (`docs/SAMS-OPS-RUNBOOK.md`) — first, quick fixes
2. **This Developer Book** (`docs/SAMS-DEVELOPER-OPS-BOOK-DENIS.md`) — deep troubleshooting
3. **Platform docs** (`DOCUMENTATION.md`) — feature reference (truncated)

Injected in `openaiEngine.ts` → `buildSystemPrompt()` as "SAMS Platform Documentation".

Optional: add sections to AI Knowledge (`ops` category) via Super Admin UI for site-specific notes.

16.4 Production URLs & paths cheat sheet

```
VPS:     ssh root@182.143.228.182
Path:    cd /var/www/sams
Health:  curl -sS http://127.0.0.1:3001/health
PM2:     pm2 status && pm2 logs sams-api --lines 30
Deploy:  bash scripts/backup-production.sh && bash scripts/deploy-production.sh
```

End of SAMS Developer & Operations Book — Denis. Maintain alongside code changes to scripts, nginx, and AI handlers.